

```
=====
## What is a Function?
=====
```

```
** A function is a block of reusable code.
```

```
** Function is used to perform a specific task.
```

```
** Instead of writing same code again and again, we can write code once inside a function and use it multiple times.
```

```
=====
## Why Functions?
=====
```

Functions are used for:

- 1) Code reusability
- 2) Code readability
- 3) Code organization
- 4) Reducing duplicate code
- 5) Easy Debugging
- 6) Easy maintenance

```
=====
## Function Syntax
=====
```

```
def function_name():
    statements
```

```
## Example :
```

```
# define function
def greet():
    print("Welcome to python")
```

```
# calling function
greet()
```

```
=====
## Important Points
=====
```

```
** def keyword is used to define a function.
```

```
** Function name should be meaningful.
```

```
** Function body should be written with indentation.
```

```
** Function will execute only when we call it.
```

```
** Function can be called multiple times.
```

**** We should use snake_case for function naming convention**

Note: snake_case means writing words in small letters and joining them with underscore _.

Ex :

```
student_name
course_fee
find_biggest
calculate_total_amount
send_email_notification
```

```
=====
## Function with Parameters
=====
```

**** Parameters are variables declared inside function brackets.**

**** They are used to receive input values into a function.**

```
---
# function with one parameter
def greet(name):
    print(name, " Welcome to python")

# calling function
greet("Ashok")
greet("John")
greet("Steve")
---

# Function with multiple parameters
def print_student_data(name, course):
    print("Student Name is :", name)
    print("Student Course is :", course)

print_student_data("Ashok", "Python")
print_student_data("John", "GEN AI")
```

```
=====
## Difference Between Parameter and Argument
=====
```

Parameter: Variable declared in function definition.

Argument: Actual value passed during function call.

Example :

```
def add(a, b):
    print(a + b)

# calling function
add(10, 20)
```

Here,

a and b are parameters.

10 and 20 are arguments.

```
=====
## Function with Return Value
=====
```

**** return keyword is used to send result back from a function.**

Example :

```
# Function with Return Value
def add(i, j):
    return i+j
```

```
result = add(10,20)
print(result)
```

```
=====
## Function Without Return Value
=====
```

**** If function does not return anything, it returns None by default.**

Example :

```
def greet():
    print("Welcome")
```

```
result = greet()
```

```
print(result)
```

Output :

```
Welcome
None
```

```
=====
## return Keyword
=====
```

**** return is used to return value from function.**

**** After return statement, function execution will stop.**

**** If we write any line of code after return statement, python will skip those lines.**

```
---
def test():
    print("Before return")
    return
    print("After return")
```

```
test()
```

Output :

```
Before return
---
```

```
=====
## Function to Check Even or Odd
=====

def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

result = check_even_odd(11)
print(f"The given number is {result}")
```

```
=====
## Function to Find Biggest Number
=====

def find_biggest(a, b):
    if a > b:
        return a
    else:
        return b

result = find_biggest(20, 35)
print(f"The biggest is {result}")
```

```
=====
## Types of Arguments
=====
```

**** Python supports different types of function arguments:**

- 1) Positional Arguments
- 2) Keyword Arguments
- 3) Default Arguments
- 4) Variable Length Arguments
- 5) Keyword Variable length Arguments

```
=====
## Positional Arguments
=====
```

=> In positional arguments, values are assigned based on position.

Positional Arguments

```
def student(name, course):
    print("Name:", name)
    print("Course:", course)

student("Ashok", "Python")
```

Important: Order is important in positional arguments.

```
=====
## Keyword Arguments
=====
```

In keyword arguments, values are passed using parameter names.

```
---
def student(name, course):
    print("Name:", name)
    print("Course:", course)

student(course="JAVA", name="John")
---
```

Important: Order is not important in keyword arguments.

```
=====
## Default Arguments
=====
```

** Default arguments are used to provide default values to parameters.

** If value is not passed, default value will be used.

```
---

## default arguments
def enroll_student(name, course="GEN AI"):
    print("Student Name:", name)
    print("Enrolled Course:", course)

enroll_student("Ashok", "DEVOPS")
enroll_student("Steve")

---
```

Note: default arguments must come after non-default arguments.

```
=====
## Variable Length Arguments
=====
```

```
add(10,20)
add(10,20,40)
add(19,78,79,797)
```

** Variable length arguments are used when we do not know how many values will be passed.

** We use *args for variable length arguments.

```
---

def print_numbers(*args):
    print(args)

print_numbers(10)
print_numbers(20, 30)
print_numbers(30, 40, 50)
```

```
---
```

Note : *args stores values in tuple format.

```
# sum of numbers using variable length args
```

```
def add_numbers(*args):
    total = 0
    for num in args:
        total = total + num
    return total

print(add_numbers(10, 20))
print(add_numbers(10, 20, 30))
print(add_numbers(10, 20, 30, 40))
```

Note: variable length argument should be the last parameter of the function.

Note-1 : After variabe length argument we can use keyword args in the function definition.

```
## Variable Length Arguments
def print_numbers(*args, model):
    print(args, model)

print_numbers(10, model="gpt")
print_numbers(20, 30, model="gpt")
print_numbers(30, 40, model="gpt")
```

```
=====
## Keyword Variable Length Arguments
=====
```

** Keyword variable length arguments are used to pass multiple key-value pairs.

=> We use **kwargs for keyword variable length arguments.

```
# keyword variable length args
```

```
def student_details(**kwargs):
    print(kwargs)

student_details(name="Ashok", course="GEN AI")
student_details(name="John", course="Python", fee=1000)
```

```
=====
## Local Variable
=====
```

** A variable declared inside a function is called local variable.

** Local variable can be used only inside that function.

```
## Example
```

```
def test():
    msg = "Hello Python"
```

```
    print(msg)
```

```
test()
```

```
## Wrong Example
```

```
def test():  
    msg = "Hello Python"  
    print(msg)
```

```
test()  
print(msg)
```

Output : NameError: name 'msg' is not defined

```
=====
```

```
## Global Variable
```

```
=====
```

** A variable declared outside a function is called global variable.

** Global variable can be used inside and outside of the function.

```
msg = "Welcome to AI Training"
```

```
def test():  
    print(msg)
```

```
def demo():  
    print(msg)
```

```
test()  
demo()  
print(msg)
```

```
=====
```

```
## global Keyword
```

```
=====
```

** global keyword is used to modify global variable inside function.

```
---
```

```
msg = "Welcome to AI Training"
```

```
def test():  
    msg = "Welcome to Python"  
    print(msg)
```

```
def demo():  
    global msg  
    msg = "Welcome to JAVA"  
    print(msg)
```

```
test()  
demo()  
print(msg)
```

```
=====
## Function Inside Function
=====
```

**** We can define one function inside another function.**

Ex-1 ::

```
def outer():
    print("outer function")

    def inner():
        print("inner function")

    inner()
```

outer()

Ex-2 :: Generate Bill Reciept for student course payment

def process_payment(student_name, course_fee, paid_amount):

```
    def validate_payment():
        if paid_amount <= 0:
            return False
        if paid_amount > course_fee:
            return False
        return True
```

```
    def calculate_balance():
        return course_fee - paid_amount
```

```
    def generate_receipt(balance):
        print("Payment Reciept")
        print("-----")
        print("Student Name : ", student_name)
        print("Course Fee : ", course_fee)
        print("Paid AMount : ", paid_amount)
        print("Balance : ", balance)
```

```
    if validate_payment():
        balance = calculate_balance()
        generate_receipt(balance)
```

```
    else:
        print("Invalid Payment")
```

process_payment(student_name="Ashok", course_fee=12000, paid_amount=5000)

Requirement : Shopping Cart Bill Calculation

When a customer places an order, we need to:

1. Calculate Total Cart Amount
2. Apply Discount
3. Add Delivery Charges

4. Generate Final Bill

If cart_amount >= 5000 then apply 2% discount

If cart_amount >= 3000 then apply 1% discount

if cart_amount >=1000 then delivery charge is 0 else delivery charge is 50 INR

=====

```
def f1(a, b):  
    return a+b
```

```
def f2(a:int, b:int) -> int:  
    return a+b
```

```
result = f2(10,20)  
print(result, type(result))
```

```
result = f2("Hi","Hello")  
print(result, type(result))
```

Note: f2 function parameters and return type data types are just hints for developes they are not mandatory to follow.

=====